claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-a6472101506f35fdc.jsonl`
- SHA-256: `b7189a65124387e628f1360fb3a2d6072c29b11f32833e898a80e5e4466d0074`
- Source modified: `2026-06-10T06:14:52+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_2a64bc88-2a8`
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`

Transcript

1. user (2026-06-10T06:12:46.622Z)

CONTEXT ? two sibling repos on a Windows machine:
- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).
- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).
Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources.
ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.
QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.
ROLE: adversarial verifier. An auditor ("eng-pipeline") claimed the finding below. Try to REFUTE it against the actual files. Default to isReal=false if the evidence does not hold up, the behavior is documented/deliberate (e.g. listed in docs/CODE_MAP.md gotchas or DEFERRED_HARDENING_PLAN.md as a known deferral ? unless the auditor adds genuinely NEW consequence), or the severity is materially overstated.
FINDING: {"title":"DB write helpers swallow all exceptions and segmenters ignore the None/False returns ? DB failures masquerade as 'fewer/no rallies' under a 'success' response","severity":"high","area":"infrastructure (DB) / segmenters","file":"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/infrastructure/database.py:112-114,126-128,162-164,176-178","evidence":"add_video/add_play_segment/add_event/add_candidate_segment/link_candidate_to_segment all catch bare Exception, log, and return False/None. In hybrid P4 (e.g. wasb_hybrid.py:212-218) a None segment_id just skips the segment silently; candidate_ids[i]=None silently drops the candidate-link. No caller anywhere checks add_video's False. SQLite is opened with no busy_timeout and no WAL (grep for busy_timeout/journal_mode/WAL: zero matches), so under any write concurrency 'database is locked' errors fire immediately and are eaten. The 'with conn:' context manager also only commits ? connections are never explicitly closed (relies on CPython refcounting).","why_it_matters":"Today a locked/full/corrupt DB mid-run yields an HTTP 200 success with partial segments and no error surface. For the committed async-job slice this becomes the primary failure mode: multiple worker threads writing to one SQLite file with zero lock handling and swallow-and-continue semantics equals silent data loss precisely when concurrency arrives. NEW (CODE_MAP says 'write helpers swallow exceptions' but not the no-WAL/no-busy_timeout posture or the unchecked-return chain to a success response)."}
Steps: open the cited file(s) and any counter-evidence files; verify quotes/line claims; judge severity honestly. Cheap greps allowed; do NOT run the pipeline. Return isReal, confidence, and

a 1-3 sentence verdict (with corrected detail if partially right).

2. user (2026-06-10T06:12:53.447Z)

```
1       import sqlite3
2       import json
3       import os
4       import logging
5       from typing import List, Dict, Any, Optional
6
7       logger = logging.getLogger(__name__)
8
9       class Database:
10          def __init__(self, db_path: str):
11              self.db_path = db_path
12              self._init_db()
13
14          def _get_connection(self):
15              conn = sqlite3.connect(self.db_path)
16              conn.row_factory = sqlite3.Row
17              # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
18              conn.execute("PRAGMA foreign_keys = ON")
19              return conn
20
21          def _init_db(self):
22              """Initializes tables for videos, play_segments, and events."""
23              with self._get_connection() as conn:
24                  cursor = conn.cursor()
25
26                  # Videos Table
27                  cursor.execute("""
28                      CREATE TABLE IF NOT EXISTS videos (
29                          id TEXT PRIMARY KEY,
30                          filepath TEXT NOT NULL,
31                          processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
32                          status TEXT NOT NULL,
33                          validation_results TEXT
34                      )
35                  """)
36
37                  # Play Segments Table (Extensible metadata JSON)
38                  cursor.execute("""
39                      CREATE TABLE IF NOT EXISTS play_segments (
40                          id INTEGER PRIMARY KEY AUTOINCREMENT,
41                          video_id TEXT NOT NULL,
42                          start_time REAL NOT NULL,
43                          end_time REAL NOT NULL,
44                          duration REAL NOT NULL,
45                          server TEXT,
46                          receiver TEXT,
47                          metadata TEXT,
48                          FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
49                      )
50                  """)
51
52                  # Events Table
53                  cursor.execute("""
54                      CREATE TABLE IF NOT EXISTS events (
55                          id INTEGER PRIMARY KEY AUTOINCREMENT,
56                          segment_id INTEGER NOT NULL,
57                          timestamp REAL NOT NULL,
58                          type TEXT NOT NULL,
59                          player TEXT,
60                          details TEXT,
61                          FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
```

```
CASCADE
                    )
                """)

                # Versioned migrations live here. PRAGMA user_version is the schema
                # contract ? bump it for every change and add an ordered `if version < N`
                # block below. Tests assert the expected version, so accidental drift fails
CI.
                version = cursor.execute("PRAGMA user_version").fetchone()[0]

                if version < 1:
                    # v1: raw candidate detections (pre-confirmation), detector-agnostic.
                    # Common fields are columns; detector-specific metrics go in `stats`
JSON,
                    # so a new segmenter reuses this table by setting its own
`detector`/`stats`.
                    cursor.execute("""
                        CREATE TABLE IF NOT EXISTS candidate_segments (
                            id INTEGER PRIMARY KEY AUTOINCREMENT,
                            video_id TEXT NOT NULL,
                            detector TEXT NOT NULL,
                            chunk_index INTEGER,
                            start_time REAL NOT NULL,
                            end_time REAL NOT NULL,
                            duration REAL NOT NULL,
                            confidence REAL,
                            stats TEXT,
                            detection_params TEXT,
                            confirmed_segment_id INTEGER,
                            human_label TEXT,
                            notes TEXT,
                            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                            FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
                            FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
ON DELETE SET NULL
                        )
                    """)
                    cursor.execute("""
                        CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
                            ON candidate_segments(video_id, detector)
                    """)
                    cursor.execute("PRAGMA user_version = 1")
                # future: if version < 2: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 2

                conn.commit()

        def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
            try:
                with self._get_connection() as conn:
                    conn.cursor().execute(
                        "INSERT OR REPLACE INTO videos (id, filepath, status,
validation_results) VALUES (?, ?, ?, ?)",
                        (video_id, filepath, status, json.dumps(validation_results))
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to add video: {e}")
                return False

        def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
```

```
            try:
                with self._get_connection() as conn:
                    cursor = conn.cursor()
                    cursor.execute(
                        "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
                        (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {}))
                    )
                    conn.commit()
                    return cursor.lastrowid
            except Exception as e:
                logger.error(f"Failed to add play segment: {e}")
                return None

        def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
            try:
                with self._get_connection() as conn:
                    conn.cursor().execute(
                        "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
                        (segment_id, timestamp, event_type, player, json.dumps(details or
{}))
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to add event: {e}")
                return False

        def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
                                  chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
                                  stats: Optional[Dict[str, Any]] = None,
                                  detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
            """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
            detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
            try:
                with self._get_connection() as conn:
                    cursor = conn.cursor()
                    cursor.execute(
                        """INSERT INTO candidate_segments
                           (video_id, detector, chunk_index, start_time, end_time, duration,
                            confidence, stats, detection_params)
                           VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
                        (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
                         confidence, json.dumps(stats or {}), json.dumps(detection_params or
{}))
                    )
                    conn.commit()
                    return cursor.lastrowid
            except Exception as e:
                logger.error(f"Failed to add candidate segment: {e}")
                return None

        def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
            """Record that a candidate detection was confirmed as a given play_segment."""
            try:
                with self._get_connection() as conn:
                    conn.cursor().execute(
```

```
                        "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id =
?",
                        (segment_id, candidate_id)
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to link candidate {candidate_id} to segment
{segment_id}: {e}")
                return False

        def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
                                     only_unlabeled: bool = False) -> List[Dict[str, Any]]:
            """Fetch candidate detections for the annotation / fine-tuning workflow.
            `stats` and `detection_params` are deserialized from JSON."""
            query = "SELECT * FROM candidate_segments WHERE video_id = ?"
            params: List[Any] = [video_id]

            if detector:
                query += " AND detector = ?"
                params.append(detector)
            if only_unlabeled:
                query += " AND human_label IS NULL"

            query += " ORDER BY start_time"

            candidates = []
            with self._get_connection() as conn:
                rows = conn.cursor().execute(query, params).fetchall()
                for row in rows:
                    d = dict(row)
                    d["stats"] = json.loads(d["stats"] or "{}")
                    d["detection_params"] = json.loads(d["detection_params"] or "{}")
                    candidates.append(d)
            return candidates

        def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
            with self._get_connection() as conn:
                row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
(video_id,)).fetchone()
                if row:
                    d = dict(row)
                    d["validation_results"] = json.loads(d["validation_results"] or "[]")
                    return d
            return None

        def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
            """
            Dynamically queries play segments based on filters:
            - duration_min: float
            - server: string
            - receiver: string
            - event_type: string (e.g. smash, foul)
            """
            query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
            params = [video_id]

            if filters.get("duration_min"):
                query += " AND r.duration >= ?"
                params.append(filters["duration_min"])

            if filters.get("server"):
                query += " AND r.server = ?"
                params.append(filters["server"])
```

```
232
233             if filters.get("receiver"):
234                 query += " AND r.receiver = ?"
235                 params.append(filters["receiver"])
236
237             if filters.get("event_type"):
238                 # Check if there is an event of a specific type inside this segment
239                 query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
240                 params.append(filters["event_type"])
241
242             segments_list = []
243             with self._get_connection() as conn:
244                 cursor = conn.cursor()
245                 rows = cursor.execute(query, params).fetchall()
246
247                 for row in rows:
248                     r_dict = dict(row)
249                     r_id = r_dict["id"]
250                     r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
251
252                     # Fetch associated events
253                     event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
254                     r_dict["events"] = []
255                     for er in event_rows:
256                         e_dict = dict(er)
257                         e_dict["details"] = json.loads(e_dict["details"] or "{}")
258                         r_dict["events"].append(e_dict)
259
260                     segments_list.append(r_dict)
261
262             return segments_list
263
264         def clear_video_data(self, video_id: str):
265             with self._get_connection() as conn:
266                 conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
267                 conn.commit()
268
```

3. user (2026-06-10T06:12:54.551Z)

```
run_phase3_eval.py:3:Walks the `manifest.json` produced by sports-data-collector's `curate
export`,
docs\CODE_MAP.md:89:- `@run_phase3_eval.py::main` ? walk manifest, segment-if-needed, score,
aggregate. `load_dotenv()` at import. ? DB key = file MD5, NOT manifest's `video_id` (kept as
label); config via `backend.config.load_config`; `segmenter_cls(db, cfg)` receives the typed
`AppConfig` (same object the live pipeline feeds segmenters).
docs\CODE_MAP.md:108:[Omitted long matching line]
docs\CODE_MAP.md:173:[Omitted long matching line]
docs\CODE_MAP.md:242:- **Reporting never breaks a run:** `emit_indexer_report` swallows all
exceptions (designed for `main.py` `finally:`) and is emitted even on validation failure.
`obsreport` is a SOFT dep ? absent -> no-op.
docs\GOLDEN_SET_VENDORING.md:44:- **Edge cases:** lets/replays, service faults, between-point
walking, coaching pauses, mid-rally net-cord, broadcast replays and graphics.
backend\eval\batch.py:3:Phase 3 of the collector->indexer bridge: walk the `manifest.json`
emitted by
docs\ROADMAP.md:53:`run_phase3_eval.py` walks the manifest, processes each video, and scores
detected
docs\PLATFORM_ARCHITECTURE.md:155:  their own VM/GPU* is a meaningful ongoing load (updates,
monitoring, firewall/NAT, the
docs\PLATFORM_ARCHITECTURE.md:506:  pool by default**, keep the training corpus a **separate,
firewalled, opt-in sub-pool**,
docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md:201:gym reverb + background
games, so audio loudness ? *court occupancy*, not *rally*. Same wall as E1 (~2.2× ceiling).
```

```
docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md:229:- **Calibration, not
representation, is the n=2 wall** ? learned probabilities separate (AUC 0.84) but the operating
```

4. user (2026-06-10T06:13:05.474Z)

```
run_process_and_stitch.py:69:        db.add_video(video_id, video_path, "processed", [])
run_phase3_eval.py:143:                db.add_video(video_id, video_path, "processed", [])
tests\test_automated_provider.py:84:    d.add_video("vid1", "/x/m.mp4", "processed", [])
tests\test_automated_provider.py:85:    d.add_play_segment("vid1", 0.0, 10.0)
tests\test_automated_provider.py:86:    d.add_play_segment("vid1", 20.0, 30.0)
tests\test_database.py:21:    vid_added = db.add_video("vid1", "vid1.mp4", "pending", [{"val":
1}])
tests\test_database.py:35:    db.add_video("vid1", "vid1.mp4", "pending", [])
tests\test_database.py:37:    sid = db.add_play_segment("vid1", 10.0, 20.0, "PlayerA",
"PlayerB", {"foo": "bar"})
tests\test_database.py:114:    db.add_video("vid1", "vid1.mp4", "processed", [])
tests\test_database.py:116:    cid = db.add_candidate_segment(
tests\test_database.py:141:    db.add_video("vid1", "vid1.mp4", "processed", [])
tests\test_database.py:143:    cid = db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
tests\test_database.py:144:    sid = db.add_play_segment("vid1", 1.0, 4.0)
tests\test_database.py:146:    assert db.link_candidate_to_segment(cid, sid) is True
tests\test_database.py:160:    db.add_video("vid1", "vid1.mp4", "processed", [])
tests\test_database.py:161:    db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
tests\test_eval_harness.py:17:    d.add_video("vid1", "/x/match.mp4", "processed", [])
tests\test_eval_harness.py:27:    db.add_play_segment("vid1", 0.0, 10.0)
tests\test_eval_harness.py:28:    db.add_play_segment("vid1", 20.0, 30.0)
tests\test_eval_harness.py:33:    db.add_candidate_segment("vid1", "yolo_hybrid", 0.0, 10.0)
tests\test_eval_harness.py:34:    db.add_candidate_segment("vid1", "tracknet_hybrid", 40.0,
50.0)
tests\test_eval_harness.py:48:    db.add_candidate_segment("vid1", "yolo_hybrid", 0.0, 10.0)
tests\test_eval_harness.py:49:    db.add_candidate_segment("vid1", "yolo_hybrid", 20.0, 30.0)
tests\test_eval_harness.py:50:    db.add_play_segment("vid1", 0.0, 10.0)
tests\test_eval_harness.py:65:    db.add_play_segment("vid1", 0.0, 10.0)
tests\test_eval_harness.py:71:    db.add_play_segment("vid1", 0.0, 10.0)
tests\test_eval_harness.py:87:    db.add_play_segment("vid1", 0.0, 10.0)
tests\test_eval_harness.py:121:    d.add_video("vid1", "/x/m.mp4", "processed", [])
tests\test_eval_harness.py:122:    d.add_play_segment("vid1", 0.0, 10.0)
tests\test_eval_regression.py:14:    d.add_video("vid1", "/x/m.mp4", "processed", [])
tests\test_eval_regression.py:16:    d.add_play_segment("vid1", 0.0, 10.0)
tests\test_eval_regression.py:17:    d.add_play_segment("vid1", 20.0, 30.0)
tests\test_eval_regression.py:56:    d.add_video("vid1", "/x/m.mp4", "processed", [])
tests\test_eval_regression.py:57:    d.add_play_segment("vid1", 0.0, 10.0)
tests\test_eval_regression.py:69:    d.add_video("vid1", "/x/m.mp4", "processed", [])
tests\test_eval_regression.py:70:    d.add_play_segment("vid1", 0.0, 10.0)
tests\test_eval_regression.py:71:    d.add_play_segment("vid1", 20.0, 30.0)
tests\test_eval_regression.py:111:    d.add_video("vid1", "/x/m.mp4", "processed", [])
tests\test_eval_regression.py:112:    d.add_play_segment("vid1", 0.0, 10.0)
tests\test_gemini.py:19:    db_mock.add_play_segment.return_value = "segment_1"
tests\test_gemini.py:58:    db_mock.add_play_segment.side_effect = ["segment_1", "segment_2"]
backend\main.py:112:            db_instance.add_video(video_id, req.video_path, "failed",
[r.dict() for r in val_results])
backend\main.py:123:        db_instance.add_video(video_id, req.video_path, "processed",
[r.dict() for r in val_results])
backend\main.py:369:        db_instance.add_video(video_id, args.video_path, status, [r.dict()
for r in val_results])
backend\infrastructure\database.py:103:    def add_video(self, video_id: str, filepath: str,
status: str, validation_results: List[Dict[str, Any]]) -> bool:
backend\infrastructure\database.py:116:    def add_play_segment(self, video_id: str, start_time:
float, end_time: float, server: Optional[str] = None, receiver: Optional[str] = None, metadata:
Optional[Dict[str, Any]] = None) -> Optional[int]:
backend\infrastructure\database.py:143:    def add_candidate_segment(self, video_id: str,
detector: str, start_time: float, end_time: float,
backend\infrastructure\database.py:166:    def link_candidate_to_segment(self, candidate_id:
int, segment_id: int) -> bool:
tests\test_tracknet_hybrid.py:60:    db.add_candidate_segment.return_value = 55
```

```
tests\test_tracknet_hybrid.py:61:    db.add_play_segment.return_value = 200
tests\test_tracknet_hybrid.py:69:    _, kwargs = db.add_candidate_segment.call_args
tests\test_tracknet_hybrid.py:72:    db.link_candidate_to_segment.assert_called_once_with(55,
200)
tests\test_tracknet_hybrid.py:127:    db.add_play_segment.return_value = 200
tests\test_tracknet_hybrid.py:132:    db.add_candidate_segment.assert_not_called()
tests\test_tracknet_hybrid.py:133:    db.link_candidate_to_segment.assert_not_called()
tests\test_tracknet_hybrid.py:150:    db.add_candidate_segment.return_value = 55
tests\test_tracknet_hybrid.py:151:    db.add_play_segment.return_value = 200
tests\test_yolo_hybrid.py:63:    db_mock.add_play_segment.return_value = "yolo_segment_1"
tests\test_yolo_hybrid.py:131:    db_mock.add_play_segment.return_value = "segment_ok"
tests\test_yolo_hybrid.py:199:    db_mock.add_candidate_segment.return_value = 77
tests\test_yolo_hybrid.py:200:    db_mock.add_play_segment.return_value = 101
tests\test_yolo_hybrid.py:222:    assert db_mock.add_candidate_segment.called
tests\test_yolo_hybrid.py:223:    _, kwargs = db_mock.add_candidate_segment.call_args
tests\test_yolo_hybrid.py:228:    db_mock.link_candidate_to_segment.assert_called_once_with(77,
101)
tests\test_yolo_hybrid.py:250:    db_mock.add_play_segment.return_value = 101
tests\test_yolo_hybrid.py:271:    db_mock.add_candidate_segment.assert_not_called()
tests\test_yolo_hybrid.py:272:    db_mock.link_candidate_to_segment.assert_not_called()
backend\pipeline\segmenters\gemini.py:418:            segment_id =
self.db.add_play_segment(video_id, start, end, None, None, metadata)
backend\pipeline\segmenters\motion.py:175:            segment_id =
self.db.add_play_segment(video_id, start, end, server, receiver, metadata)
backend\pipeline\segmenters\tracknet_hybrid.py:164:                candidate_ids[i] =
self.db.add_candidate_segment(
backend\pipeline\segmenters\tracknet_hybrid.py:238:                segment_id =
self.db.add_play_segment(video_id, start, end, None, None, metadata)
backend\pipeline\segmenters\tracknet_hybrid.py:242:
self.db.link_candidate_to_segment(candidate_id, segment_id)
backend\pipeline\segmenters\wasb_hybrid.py:145:                candidate_ids[i] =
self.db.add_candidate_segment(
backend\pipeline\segmenters\wasb_hybrid.py:212:                segment_id =
self.db.add_play_segment(video_id, start, end, None, None, metadata)
backend\pipeline\segmenters\wasb_hybrid.py:216:
self.db.link_candidate_to_segment(candidate_id, segment_id)
backend\pipeline\segmenters\yolo_hybrid.py:466:                candidate_ids[i] =
self.db.add_candidate_segment(
backend\pipeline\segmenters\yolo_hybrid.py:551:                segment_id =
self.db.add_play_segment(video_id, start, end, None, None, metadata)
backend\pipeline\segmenters\yolo_hybrid.py:558:
self.db.link_candidate_to_segment(candidate_id, segment_id)
```

5. user (2026-06-10T06:13:05.842Z)

```
180                     os.remove(w_path)
181                 except OSError:
182                     pass
183                 return valid
184
185             ai_segments: List[dict] = []
186             with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
187                 futures = [executor.submit(process_candidate_window, wi, cw)
188                            for wi, cw in enumerate(all_candidate_windows)]
189                 for future in futures:
190                     ai_segments.extend(future.result())
191             try:
192                 os.rmdir(handoff_dir)
193             except OSError:
194                 pass
195             logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
196
197             # --- Phase 4: DB population (shared pattern) ---
```

```
198             segments_data = []
199             for segment in ai_segments:
200                 try:
201                     start = float(segment["start_time"])
202                     end = float(segment["end_time"])
203                     dur = end - start
204                     shot_count = segment.get("shuttle_exchange_count")
205                     try:
206                         shot_count = int(shot_count)
207                     except (ValueError, TypeError):
208                         shot_count = max(3, int(dur / 0.8))
209                     metadata = {"shot_count": shot_count,
210                                 "shot_count_confidence":
segment.get("shot_count_confidence", "Unknown"),
211                                 "detector": f"wasb-hybrid-{model_name}"}
212                     segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
213                     if segment_id:
214                         candidate_id = segment.get("_candidate_id")
215                         if candidate_id is not None:
216                             self.db.link_candidate_to_segment(candidate_id, segment_id)
217                         segments_data.append({"id": segment_id, "start_time": start,
"end_time": end,
218                                               "duration": dur, "metadata": metadata})
219                 except Exception as e:
220                     logger.warning("Skipping invalid segment struct: %s", e)
221
222             return segments_data
223
224
225     segmenter_registry.register(WasbHybridSegmenter)
226
```

6. user (2026-06-10T06:13:16.674Z)

```
3:**Purpose:** cold-start navigation map for the badminton-highlight-indexer ? a self-hosted
video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a
segmenter-agnostic eval/calibration stack, typed config, and per-video observability reporting.
33:  -> finally: emit_indexer_report(...)    @backend/reporting/__init__.py  (? ALWAYS runs ?
even on validation failure / exception; NEVER raises; grafts response["reports"])
108:[Omitted long matching line]
173:[Omitted long matching line]
176:[Omitted long matching line]
242:- **Reporting never breaks a run:** `emit_indexer_report` swallows all exceptions (designed
for `main.py` `finally:`) and is emitted even on validation failure. `obsreport` is a SOFT dep ?
absent -> no-op.
324:| `test_validators_base.py` | `pipeline/validators/base.ValidationRegistry`
(register/run_all/`run_all_with_context` -> `(results,context)`; exception captured not raised)
|
```

7. user (2026-06-10T06:13:18.466Z)

```
62:eval/regression set. Leakage would silently inflate the very metrics the product
102:- Persist job state in the existing SQLite DB via the migration ladder
108:- **SQLite thread-safety:** verify `check_same_thread`/connection-per-thread;
```

8. user (2026-06-10T06:13:25.145Z)

```
100       - `::load_config` ? precedence built-in-defaults -> file overrides. ? **shallow top-
level merge** (`{**defaults, **file_data}`): a section in config.json REPLACES the whole
defaults dict for that section (Pydantic then re-applies field defaults for missing leaves).
Missing/unreadable/parse-error file -> fully-defaulted AppConfig + `[CONFIG WARNING]` (non-
fatal).
101       - `::save_default_config` ? writes fresh `config.json`, **overwrites** if present
```

(used by setup.py/`--setup`). `::get_default_config` ? ? transitional plain-dict alias.
102   - `@config.json` ? on-disk config; mirrors `AppConfig`. ?
`indexing.default_segmenter='gemini'` diverges from model default `'yolo_hybrid'` ? file
wins at runtime.
103   - `@setup.py` ? `::init_default_config` ? delegates to
`backend.config.save_default_config` (single-source defaults; skips if config.json exists).
`::run_pip_install` GPU-aware (`nvidia-smi` -> cu124 else cpu). `::run_diagnostics` (Python?3.8,
ffmpeg/ffprobe on PATH, optional torch+CUDA).
104
105   ### 2.1 Segmenters (under pipeline/segmenters) ?
106   - `@backend/pipeline/segmenters/base.py`
107   - `::BasePlaySegmenter` ABC ? `__init__(db, config)`; abstract
`name`,`description`,`process_video`. § `process_video(video_id, video_path, player_pool=None,
max_frames=None) -> List[dict{id,start_time,end_time,duration,metadata}]`.
108   - `::SegmenterRegistry`, `::segmenter_registry` (singleton). ? **register a new
segmenter**: subclass `BasePlaySegmenter`, call `segmenter_registry.register(MyCls)` at module
bottom, AND import the module in `@backend/pipeline/segmenters/__init__.py` (registration is an
IMPORT side-effect). ? `register()` builds `cls(None, {})` just to read `.name` ->
name/description/`__init__` MUST NOT touch `self.db`/`self.config`. ? `list_available()` re-
instantiates + swallows exceptions into "No description available." (a throwing `description` is
masked).
109   - `@backend/pipeline/segmenters/__init__.py` ? imports all 5
(`motion,gemini,yolo_hybrid,tracknet_hybrid,wasb_hybrid`) = the de-facto registration manifest;
re-exports them + `segmenter_registry` via `__all__`.
110   - `@backend/pipeline/segmenters/wasb_hybrid.py::WasbHybridSegmenter` (name
`'wasb_hybrid'`) ? WSL WASB substrate (MIT; ships pretrained badminton weights, the LIVE
default). Imports `WasbRunner`/`WasbConfig` from `pipeline/detectors/wasb_runner`. Config
`indexing.wasb.velocity_threshold(0.02)` + shared
`dead_time_merge_gap/min_action_window_duration/ai_handoff_padding/ai_max_workers`. ? whole-
video single pass; reuses YOLO-named keys `log_yolo_candidates/store_yolo_candidates`; needs
`{PROVIDER}_API_KEY` (missing -> []); `detection_params` has NO `queue_length` (unlike
tracknet). ? divergent shot_count: `int(shot_count)` inside try with no `is None` short-circuit
(copy-paste drift).
111   - `@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` (name
`'tracknet_hybrid'`) ? copy-paste twin of wasb_hybrid; imports `TrackNetRunner`/`TrackNetConfig`
from `pipeline/detectors/tracknet_runner`; `indexing.tracknet.velocity_threshold(0.02)`;
`detection_params` adds `queue_length`. ? any P3/P4 fix must be applied to BOTH (and arguably
yolo_hybrid).
112   - `@backend/pipeline/segmenters/yolo_hybrid.py::HybridYoloSegmenter` (name
`'yolo_hybrid'`, no YOLO remains ? torchvision detector + supervision ByteTrack; ADR-005).
`::_PERSON_CLASS_ID=1` ? (ultralytics used 0; wrong value -> zero detections).
`::_lazy_load_model` (lazy `torchvision` import so module/tests load without it),
`::_extract_action_windows_from_chunk`, `::_make_tracker` (per-chunk ByteTrack ? track IDs
chunk-local), `::_DETECTOR_FACTORIES`/`::_DEFAULT_DETECTOR='fasterrcnn_resnet50_fpn'`. ?
velocity at `effective_fps = fps/frame_skip` -> `yolo_velocity_threshold` coupled to
`frame_skip`; pipelined-vs-sequential P2 (prefetch ffmpeg overlaps sequential GPU);
`yolo_tracker`/`bytetrack.yaml` is dead/back-compat. ? `gemini_*` config keys are deprecated
aliases still honored.
113   - `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter` (name `'gemini'`) ?
pure-AI, no P2. ? chunks only if `duration>chunk_duration`; `max_workers=5` hardcoded;
`max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates; `parse_time` accepts
colon timestamps (warns); does NOT persist candidate segments (only hybrids feed the fine-tuning
table).
114   - `@backend/pipeline/segmenters/motion.py::MotionPlaySegmenter` (name `'motion'`) ?
pixel absdiff baseline. ? **metadata + serve/smash/foul/winner events are RANDOM**
(random.sample/randint/uniform/choice); only segmenter using `db.add_event` and passing
server/receiver; uses `print()`; honors `max_frames`.

9. user (2026-06-10T06:13:26.591Z)

168   - `@backend/annotations/file_provider.py::FileProvider` (`'file'`; job_id IS the
resolved CSV path; `_resolve` -> `<csv_dir>/<video_ref>.csv`; ->
`backend.eval.annotations.load_annotations`; ? `guideline`/`sport` ignored).
169   - `@backend/annotations/automated_provider.py::AutomatedProvider` (`'auto'`; GS-4 stub;
injectable `::Labeler`, in-process `_jobs` store, `::warn_if_same_lineage` ORACLE RULE ? only

warns, does not block; default `_not_configured` RAISES NotImplementedError). ? get via `annotation_registry.get('auto', labeler=fn, lineage='...')`; registered as `"auto"` only because no-arg ctor succeeds.
170   - `@backend/annotations/__init__.py` ? ? `noqa F401` imports of file_provider/automated_provider are load-bearing (removing de-registers providers).
171
172   ### 2.7 Infrastructure (DB) ?
173   - `@backend/infrastructure/database.py::Database` ? SQLite, 4 tables (`videos/play_segments/events/candidate_segments`), `PRAGMA user_version` migration ladder in `::_init_db` (currently =1, `version<1` creates `candidate_segments`). `::_get_connection` (? re-issues `PRAGMA foreign_keys=ON` per connection ? else CASCADE/SET NULL silently skipped). Methods: `add_video`,`add_play_segment`,`add_event`,`add_candidate_segment`,`link_candidate_to_segment`,`query_candidate_segments(detector=,only_unlabeled=)`,`query_play_segments`,`get_video`, `clear_video_data`. ? candidate_segments is the detector-agnostic fine-tuning extension point (common fields columns, detector-specifics in `stats`/`detection_params` JSON; index `idx_candidates_video_detector`). ? `add_video` is `INSERT OR REPLACE` -> cascade-DELETES dependent segments on re-ingest; events column is `type` not `event_type`; `query_play_segments` filters use truthiness so `duration_min=0`/empty-string = "no filter"; write helpers swallow exceptions.
174
175   ### 2.8 Reporting / observability ? (per-video, soft-dep `obsreport`)
176   - `@backend/reporting/__init__.py` ? `::emit_indexer_report(*, db, video_id, video_path, config, val_results, val_context=None, segmenter_requested=, segmenter_actual=, was_reingest=False, segmentation_ran=True)` ? single entrypoint, keyword-only. Normalizes typed config (`config.model_dump(...)` if `hasattr model_dump`); short-circuits to None if unavailable/disabled; pulls play_segments/candidates from `db` (read-only `query_play_segments(vid,{})`/`query_candidate_segments(vid)`); delegates to `harvest.record_run`; writes via `rec.write(output_dir=config["report"]["output_dir"])`. ? wraps everything in bare `except Exception: return None` ? **NEVER raises** (designed for `main.py`'s `finally:`); writes `<video>.report.json`/`.report.md`/`.summary.md` next to video. `::OBSREPORT_AVAILABLE` ? soft-dep gate (True only if `import obsreport` AND `from . import harvest` both succeed). `::report_enabled(config)` reads `config["report"]["enabled"]` default True.
177   - `@backend/reporting/harvest.py` ? records raw FACTS into `obsreport.RunRecorder` (the footgun RULES live in spec.yaml, not here). `::record_run(...)` ? top-level assembler (does NOT write); builds RunRecorder, reads `config_default = config["indexing"]["default_segmenter"]`, runs 4 stage builders + `_highlights`, `rec.finalize()`. `::_AI_SEGMENTERS={yolo_hybrid,tracknet_hybrid,wasb_hybrid,gemini}` ? (add new AI segmenters here). `::video_meta_from_context` (derives VideoMeta from `val_context['ffprobe_meta']`; now also `audio_present` for Input Quality reports extension; § `fps_source ? {unknown,probed,fallback}`), `::_validation_stage`, `::_segmentation_stage` (§ `details.{segmenter_requested,segmenter_actual,config_default,matches_config_default,segmenter_explicitly_requested,was_reingest,detector,metadata_source}`; ? marks `motion` output "synthetic/heuristic"), `::_ai_stage` (§ `details.{ai_based,provider,model,api_key_present}`; metric `confirmed_segments`), `::_highlights` (coverage_pct). ? `record_run` fps fallback: `fps_source=='unknown'` + ran -> promotes to "fallback", forces fps=30.0 (mirrors runners). `::_SPEC_PATH`/`::load_spec` resolve `spec.yaml` next to module. (Soft import of obsreport so pure helpers like video_meta are importable for tests/coverage even without the dep.)
178   - `@backend/reporting/spec.yaml` ? § declarative report layout + footgun engine consumed by obsreport. `::customer_verdict` (pass/pass_with_warnings/fail messages). `::sections` (ordered `{id,title,kind,audience?{both,customer,technical}}`). ? `::rules` ? each `{code,severity,when[{path,op,value}],message,faq_ref,customer_message?}`; `when` clauses AND'd; `path` dotted-navigates the recorder JSON. ? `path` strings are a HARD coupling to harvest's stage/metric/detail names ? renaming a detail in harvest.py silently breaks the matching rule. Rules: `silent_provider_noop`(error: no api key + 0 segs), `ai_empty_vs_no_rallies`(warn), `fps_fallback_used`(warn), `synthetic_motion_metadata`(warn), `segmenter_divergence`(info), `reingest_replaced_prior`(info), `validation_failed`(error: halts indexing).
179
180   ---
181
182   ## 3. DATA CONTRACTS (canonical shapes ? one place)

10. user (2026-06-10T06:13:27.123Z)

80   - Small corpus ? tiny held-out set ? high metric variance; the guardrail is correct

```
81        but the numbers will be noisy until the golden set grows.
82      - MD5-only dedup is insufficient (re-encodes evade it) ? must track match identity.
83
84      ### Verification
85      - Unit test: overlapping train/eval ids ? `assert_disjoint` raises.
86      - Integration: a deliberately-leaked manifest fails the training entrypoint.
87
88      ---
89
90      ## 3. Async job model (debt #16)
91
92      ### Problem
93      `POST /api/process` runs validate ? segment ? report **inline**, blocking the
94      request for the full duration of a long video. Unworkable as a hosted/product API.
95
96      ### Approach (minimal first slice ? single self-hosted box)
97      - `POST /api/process` returns **202 + `job_id`**; add `GET /api/jobs/{job_id}`
98        returning `{status, progress, result, reports}`.
99      - Run the pipeline on a background worker ? start with a `ThreadPoolExecutor`
100       (segmenters already use threads internally for AI handoff, so this fits); leave a
101       real queue (RQ/Celery) for later if multi-process is needed.
102     - Persist job state in the existing SQLite DB via the migration ladder
103       (`backend/infrastructure/database.py::_init_db`, bump `PRAGMA user_version` 1 ? 2,
104       add a `jobs` table). The observability report becomes the job's artifact.
105     - Update the static UI to poll `GET /api/jobs/{id}` instead of blocking.
106
107     ### Risks
108     - **SQLite thread-safety:** verify `check_same_thread`/connection-per-thread;
109       `_get_connection` already re-issues `PRAGMA foreign_keys=ON` per connection ?
110       confirm it's safe under the executor.
111     - Cancellation/timeout and crash recovery (a job in RUNNING after a restart).
112     - In-process store is not horizontally scalable ? acceptable for the current
113       single-box self-hosted target; call it out so it isn't mistaken for production-HA.
114
115     ### Verification
116     - Submit a job ? poll ? DONE with the same segments the sync path produced.
117     - Server restart mid-job leaves a recoverable/clearly-FAILED job, not a hang.
118
119     ---
120
121     ## Out of this doc
122     - **Detector/WSL abstraction (#6):** in flight (PR #49).
123     - **Unify hybrid segmenters (#7 in LOW_REGRET):** owner-deferred.
124     - **ByteTrack `supervision<0.30.0` migration + real-footage validation (#14/#15):**
125       need a GPU/WSL box and real video ? owner-machine work, tracked separately.
126
```

11. user (2026-06-10T06:13:56.599Z)

```
95
96          video_id = compute_video_id(req.video_path)
97          was_reingest = db_instance.get_video(video_id) is not None
98
99          # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
100         logger.info(f"Running validations for {req.video_path}...")
101         val_results, val_context = registry.run_all_with_context(req.video_path,
global_config)
102         failures = [r for r in val_results if not r.passed]
103
104         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
105         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
106         seg_name = req.segmenter or default_seg
107         segmenter_actual = None
```

```
108         segmentation_ran = False
109         response: Optional[Dict[str, Any]] = None
110         try:
111             if failures:
112                 db_instance.add_video(video_id, req.video_path, "failed", [r.dict() for r in
val_results])
113                 response = {
114                     "video_id": video_id,
115                     "status": "failed",
116                     "message": "Video failed validation checks.",
117                     "results": [r.dict() for r in val_results],
118                 }
119                 return response
120 
121             # 2. Run segmenter & indexer
122             print(f"[API] Video passed checks. Segmenting and indexing...")
123             db_instance.add_video(video_id, req.video_path, "processed", [r.dict() for r in
val_results])
124 
125             segmenter_cls = segmenter_registry.get(seg_name)
126             if not segmenter_cls:
127                 available = list(segmenter_registry.list_available().keys())
128                 raise HTTPException(
129                     status_code=400,
130                     detail=f"Segmenter '{seg_name}' not found. Available segmenters:
{available}"
131                 )
132 
133             print(f"[API] Using segmenter: {seg_name}")
134             segmenter_actual = seg_name
135             segmenter = segmenter_cls(db_instance, global_config)
136             result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
137             segmentation_ran = True
138 
139             if isinstance(result, Failure):
140                 print(f"[API] Segmenter failed: {result.message}")
141                 response = {
142                     "video_id": video_id,
143                     "status": "failed",
144                     "message": f"Segmenter '{seg_name}' failed: {result.message}",
145                     "results": [r.dict() for r in val_results],
146                     "play_segments": [],
147                 }
148                 return response
149 
150             segments = result.value if hasattr(result, "value") else result
151 
152             response = {
153                 "video_id": video_id,
154                 "status": "success",
```

12. user (2026-06-10T06:13:57.179Z)

```
100                 "velocity_thresh": velocity_thresh,
101                 "merge_gap": merge_gap,
102                 "min_action_window_duration": min_window_duration,
103                 "weights_path": wasb_cfg.weights_path,
104             }
105 
106             ok, msg = runner.healthcheck()
107             if not ok:
108                 logger.error("WASB WSL environment not ready: %s", msg)
109                 return []
110             logger.info("WASB WSL env OK: %s", msg)
```

```
111
112             # --- Phase 2: shuttle trajectory -> candidate rally windows ---
113             t_start = time.time()
114             out_dir = os.path.join("output", "wasb")
115             csv_path = runner.run_predict(video_path, out_dir)
116             if not csv_path:
117                 logger.error("WASB produced no trajectory; aborting.")
118                 return []
119
120             points = runner.parse_trajectory_csv(csv_path)
121             logger.info("Parsed %d trajectory points (%d visible).",
122                         len(points), sum(1 for p in points if p.visible))
123
124             all_candidate_windows = runner.trajectory_to_action_windows(
125                 points, fps=fps, frame_width=frame_width, chunk_start=0.0,
126                 velocity_thresh=velocity_thresh, merge_gap=merge_gap,
127                 min_window_duration=min_window_duration, chunk_index=0,
128             )
129             logger.info("Phase 2 (WASB) completed in %.1fs. Candidate windows: %d",
130                         time.time() - t_start, len(all_candidate_windows))
131
132             if log_candidates:
133                 for cw in all_candidate_windows:
134                     logger.info(f"[WASB rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
135                                 f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
136                                 f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")
137
138             candidate_ids = [None] * len(all_candidate_windows)
139             if store_candidates:
140                 for i, cw in enumerate(all_candidate_windows):
141                     stats = {
142                         "peak_velocity": cw["peak_velocity"], "mean_velocity":
cw["mean_velocity"],
143                         "active_frame_count": cw["active_frame_count"], "track_id_count":
cw["track_id_count"],
144                     }
145                     candidate_ids[i] = self.db.add_candidate_segment(
146                         video_id, detector="wasb_hybrid", start_time=cw["start"],
end_time=cw["end"],
147                         chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"]),
148                         stats=stats, detection_params=detection_params,
149                     )
150
151             if not all_candidate_windows:
152                 return []
153
154             # --- Phase 3: AI provider handoff (shared pattern) ---
155             handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
156             os.makedirs(handoff_dir, exist_ok=True)
157             logger.info("Phase 3: %s validation on %d candidate windows...",
158                         provider_name, len(all_candidate_windows))
159
160             def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
161                 w_start = max(0, window["start"] - handoff_padding)
162                 w_end = min(video_duration, window["end"] + handoff_padding)
163                 w_dur = w_end - w_start
164                 w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
165                 if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
166                     return []
167                 prompt = sport_profile.get_prompt(chunk_duration=w_dur)
168                 segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
```

```
169                                                 label=f"Handoff {wi+1}")
170                 valid = []
171                 for seg in segments:
172                     try:
173                         seg["start_time"] = float(seg["start_time"]) + w_start
174                         seg["end_time"] = float(seg["end_time"]) + w_start
175                         seg["_candidate_id"] = candidate_ids[wi]
176                         valid.append(seg)
177                     except (ValueError, TypeError, KeyError) as e:
178                         logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
179                 try:
```

13. user (2026-06-10T06:14:46.805Z)

```
Structured output provided successfully
```

14. assistant (2026-06-10T06:14:52.271Z)

```
Verification complete.
```